```
# ============================================================================
# STEP 1: Load the RData files
# ============================================================================
!pip install -q pyreadr gender-guesser

import pyreadr
import pandas as pd
import numpy as np
import re
from datetime import datetime
from scipy import stats
import gender_guesser.detector as gender
from google.colab import files

uploaded = files.upload()   # upload senator_tweets.RData and senator_wiki.RData

loaded = {}
for fname in [f for f in uploaded.keys() if f.endswith('.RData')]:
    result = pyreadr.read_r(fname)
    for obj_name, df in result.items():
        loaded[obj_name] = df

senator_tweets = loaded['senator_tweets'].reset_index(drop=True)
senator_wiki   = loaded['senator_wiki'].reset_index(drop=True)

print(f"senator_tweets: {senator_tweets.shape}")
print(f"senator_wiki:   {senator_wiki.shape}")
```

```
Choose Files 2 files
senator_tweets.RData(n/a) - 432527 bytes, last modified: 3/9/2026 - 100% done
senator_wiki.RData(n/a) - 39139 bytes, last modified: 3/9/2026 - 100% done
Saving senator_tweets.RData to senator_tweets (3).RData
Saving senator_wiki.RData to senator_wiki (3).RData
senator_tweets: (2401, 7)
senator_wiki:   (84, 3)
```

```
senator_tweets = loaded['senator_tweets']
senator_wiki = loaded['senator_wiki']

print(senator_wiki.iloc[0]['wiki_text'])
```

```
James Moore is an American politician serving as a United States Senator from FL. A member of the Democrat Party, James Moor
```

```
# ============================================================================
# STEP 2: Parse structured fields out of wiki_text, infer gender from name
# ============================================================================
def extract_state(text):
    m = re.search(r'Senator from ([A-Z]{2})', str(text))
    return m.group(1) if m else np.nan

def extract_party(text):
    m = re.search(r'member of the (\w+) Party', str(text))
    return m.group(1) if m else np.nan

def extract_birth_year(text):
    m = re.search(r'born in (\d{4})', str(text))
    return int(m.group(1)) if m else np.nan

senator_wiki['state']      = senator_wiki['wiki_text'].apply(extract_state)
senator_wiki['party']      = senator_wiki['wiki_text'].apply(extract_party)
senator_wiki['birth_year'] = senator_wiki['wiki_text'].apply(extract_birth_year)

# First name comes from the wikipedia_id, e.g. 'James_Moore' -> 'James'
senator_wiki['first_name'] = senator_wiki['wikipedia_id'].astype(str).str.split('_').str[0]

d = gender.Detector(case_sensitive=False)
GENDER_MAP = {
    'male': 'M', 'mostly_male': 'M',
    'female': 'F', 'mostly_female': 'F',
    'andy': np.nan, 'unknown': np.nan
}
senator_wiki['gender_raw'] = senator_wiki['first_name'].apply(d.get_gender)
senator_wiki['gender'] = senator_wiki['gender_raw'].map(GENDER_MAP)

CURRENT_YEAR = datetime.now().year
senator_wiki['age'] = CURRENT_YEAR - senator_wiki['birth_year']

print("\nGender breakdown:")
print(senator_wiki['gender'].value_counts(dropna=False))
```

```
print("\nSample parsed rows:")
print(senator_wiki[['bioguide_id','first_name','gender','state','party','birth_year','age']].head(10))
```

```
Gender breakdown:
gender
M    43
F    41
Name: count, dtype: int64

Sample parsed rows:
         bioguide_id first_name gender state       party  birth_year  age
rownames
1            B001000      James      M    FL    Democrat        1951   75
2            B001001   Patricia      F    IA    Democrat        1977   49
3            B001002      Sarah      F    AL  Republican        1955   71
4            B001003       Mark      M    CT    Democrat        1967   59
5            B001004      Brian      M    MO    Democrat        1950   76
6            B001006     Steven      M    MS    Democrat        1955   71
7            B001008       Mark      M    AZ    Democrat        1965   61
8            B001009    Jessica      F    TN    Democrat        1974   52
9            B001010   Michelle      F    MT  Republican        1968   58
10           B001011       John      M    HI    Democrat        1972   54
```

```
# ============================================================================
# QUESTION (a) — How many senators' Twitter accounts were found?
# ----------------------------------------------------------------------------
# NOTE: these are synthetic/sample handles, not live Twitter accounts, so we
# can't call the real Twitter API here. "Found on Twitter" is interpreted as:
# the senator has at least one tweet present in senator_tweets (i.e. their
# account was successfully located and scraped when this dataset was built).
# ============================================================================
tweeted_ids = set(senator_tweets['bioguide_id'].unique())
senator_wiki['found_on_twitter'] = senator_wiki['bioguide_id'].isin(tweeted_ids)

n_total = len(senator_wiki)
n_found = int(senator_wiki['found_on_twitter'].sum())
print(f"\n(a) {n_found} out of {n_total} senators' Twitter accounts were found (have tweet data).")
```

```
(a) 77 out of 84 senators' Twitter accounts were found (have tweet data).
```

```
# ============================================================================
# QUESTION (b) — Average number of published tweets by gender
# ============================================================================
tweet_counts = senator_tweets.groupby('bioguide_id').size().rename('tweet_count')
wiki_with_counts = senator_wiki.merge(tweet_counts, on='bioguide_id', how='left')
wiki_with_counts['tweet_count'] = wiki_with_counts['tweet_count'].fillna(0)

avg_tweets_by_gender = wiki_with_counts.groupby('gender')['tweet_count'].mean()
print(f"\n(b) Average number of published tweets by gender:")
print(avg_tweets_by_gender)
```

```
(b) Average number of published tweets by gender:
gender
F    23.365854
M    23.186047
Name: tweet_count, dtype: float64
```

```
# ============================================================================
# QUESTION (c) — Correlation between senator age and "account creation"
# ----------------------------------------------------------------------------
# NOTE: no real account-creation timestamp exists in this dataset. As a proxy,
# we use each senator's EARLIEST tweet timestamp in senator_tweets.
# ============================================================================
senator_tweets['created_at'] = pd.to_datetime(senator_tweets['created_at'])
earliest_tweet = senator_tweets.groupby('bioguide_id')['created_at'].min().rename('account_proxy_created_at')

wiki_with_dates = senator_wiki.merge(earliest_tweet, on='bioguide_id', how='inner')
wiki_with_dates['account_age_days'] = (
    pd.Timestamp.now() - wiki_with_dates['account_proxy_created_at']
).dt.days

valid = wiki_with_dates.dropna(subset=['age', 'account_age_days'])
corr, p_value = stats.pearsonr(valid['age'], valid['account_age_days'])
print(f"\n(c) Correlation between senator age and (proxy) account age:")
print(f"    r = {corr:.3f}, p = {p_value:.3f}, n = {len(valid)}")
```

```
(c) Correlation between senator age and (proxy) account age:
```

```
    r = -0.235, p = 0.039, n = 77
```

```
# ============================================================================
# QUESTION (d) — Last two tweets per senator; avg retweets by gender
# ============================================================================
senator_tweets_sorted = senator_tweets.sort_values(['bioguide_id', 'created_at'], ascending=[True, False])
last_two = senator_tweets_sorted.groupby('bioguide_id').head(2)

last_two_with_gender = last_two.merge(
    senator_wiki[['bioguide_id', 'gender']], on='bioguide_id', how='left'
)

avg_retweets_by_gender = last_two_with_gender.groupby('gender')['retweet_count'].mean()
print(f"\nCollected {len(last_two_with_gender)} tweets (last 2 per senator, {last_two_with_gender['bioguide_id'].nunique()}
print(f"\n(d) Average retweets received, by gender of the publishing senator:")
print(avg_retweets_by_gender)
```

```
Collected 184 tweets (last 2 per senator, 92 senators)

(d) Average retweets received, by gender of the publishing senator:
gender
F    33.337838
M    35.487500
Name: retweet_count, dtype: float64
```

```
# ============================================================================
# SUMMARY
# ============================================================================
print("\n" + "=" * 60)
print("SUMMARY OF RESULTS")
print("=" * 60)
print(f"a) Accounts found on Twitter : {n_found} / {n_total}")
print(f"b) Avg tweets by gender      :\n{avg_tweets_by_gender.to_string()}")
print(f"c) Age vs account-age corr   : r={corr:.3f}, p={p_value:.3f}")
print(f"d) Avg retweets by gender    :\n{avg_retweets_by_gender.to_string()}")
```

```
============================================================
SUMMARY OF RESULTS
============================================================
a) Accounts found on Twitter : 77 / 84
b) Avg tweets by gender      :
gender
F    23.365854
M    23.186047
c) Age vs account-age corr   : r=-0.235, p=0.039
d) Avg retweets by gender    :
gender
F    33.337838
M    35.487500
```